

EXP-7

Student Name: ANSHAJ CHAUHAN

UID: 24BAI70503

Branch: AIT-CSE (AIML)

Section/Group: 24AIT_KRG-1A

Semester: 5

Subject Code: 24CSP-310

Subject Name : Advanced Database Management System

Q1) Consider a relation R having attributes as R(ABCD), functional dependencies are given below:

$AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$

Identify the set of candidate keys possible in relation R. List all the set of prime and non prime attributes.

Ans:

Functional Dependencies: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$

- Attribute Closure Analysis:
 - Attribute 'B' is not present on the right-hand side of any functional dependency, so it must be part of every candidate key.
 - Closure of AB: $(AB)^+ = \{A, B, C, D\}$. AB is a candidate key.
 - Since $D \rightarrow A$, substitute A with D in AB to test BD: $(BD)^+ = \{B, D, A, C\} = \{A, B, C, D\}$. BD is a candidate key.
 - Since $C \rightarrow D$, substitute D with C in BD to test BC: $(BC)^+ = \{B, C, D, A\} = \{A, B, C, D\}$. BC is a candidate key.
- Candidate Keys: $\{AB, BC, BD\}$
- Prime Attributes: A, B, C, D
- Non-Prime Attributes: None (or $\emptyset$)

Q2) Relation R(ABCDE) having functional dependencies as :

$A \rightarrow D$, $B \rightarrow A$, $BC \rightarrow D$, $AC \rightarrow BE$

Identify the set of candidate keys possible in relation R. List all the set of prime and non prime attributes.

ANS:

Functional Dependencies: $A \rightarrow D$, $B \rightarrow A$, $BC \rightarrow D$, $AC \rightarrow BE$

Attribute Closure Analysis:

- Attribute 'C' is not present on the right-hand side of any functional dependency, so it must be part of every candidate key.
- Closure of BC: $(BC)^+ = \{B, C, A, D, E\}$ (Using $B \rightarrow A$ gives A, $A \rightarrow D$ gives D, $AC \rightarrow BE$ gives E). BC is a candidate key.
- Since $B \rightarrow A$, we can check if replacing B with A in BC yields a key. Closure of AC: $(AC)^+ = \{A, C, B, E, D\}$ (Using $AC \rightarrow BE$ gives B, E; $A \rightarrow D$ gives D). AC is a candidate key.

Candidate Keys: $\{AC, BC\}$

Prime Attributes: A, B, C

Non-Prime Attributes: D, E

Q3) Consider a relation R having attributes as R(ABCDE), functional dependencies are given below:

$B \rightarrow A$, $A \rightarrow C$, $BC \rightarrow D$, $AC \rightarrow BE$

Identify the set of candidate keys possible in relation R. List all the set of prime and non prime attributes.

ANS:

Functional Dependencies: $B \rightarrow A$, $A \rightarrow C$, $BC \rightarrow D$, $AC \rightarrow BE$

- Attribute Closure Analysis:
 - Let's test single attributes since the dependencies chain deeply.
 - Closure of A: $(A)^+ = \{A, C, B, E, D\}$ (Using $A \rightarrow C$ gives C, then $AC \rightarrow BE$ gives B, E. With B and C now present, $BC \rightarrow D$ gives D). A is a candidate key.
 - Closure of B: $(B)^+ = \{B, A, C, E, D\}$ (Using $B \rightarrow A$ gives A. Since A is a candidate key, B can also determine all attributes). B is a candidate key.
 - No other single attribute (C, D, or E) can determine the full relation, and any combination including A or B would be a superkey, not a minimal candidate key.
- Candidate Keys: $\{A, B\}$
- Prime Attributes: A, B
- Non-Prime Attributes: C, D, E

Q4) Consider a relation R having attributes as R(ABCDEF), functional dependencies are given below:

$A \rightarrow BCD$, $BC \rightarrow DE$, $B \rightarrow D$, $D \rightarrow A$

Identify the set of candidate keys possible in relation R. List all the set of prime and non prime attributes.

ANS: Functional Dependencies: $A \rightarrow BCD$, $BC \rightarrow DE$, $B \rightarrow D$, $D \rightarrow A$

- Attribute Closure Analysis:
 - Attribute 'F' does not appear in any functional dependency. It cannot be derived, so 'F' must be part of every candidate key.
 - Closure of AF: $(AF)^+ = \{A, F, B, C, D, E\}$ (Using $A \rightarrow BCD$ gives B, C, D; then $BC \rightarrow DE$ gives E). AF is a candidate key.
 - Since $D \rightarrow A$, substitute A with D to test DF: $(DF)^+ = \{D, F, A, B, C, E\}$ (Using $D \rightarrow A$ gives A, then A determines the rest). DF is a candidate key.
 - Since $B \rightarrow D$, substitute D with B to test BF: $(BF)^+ = \{B, F, D, A, C, E\}$ (Using $B \rightarrow D$ gives D, then D determines the rest). BF is a candidate key.
- Candidate Keys: {AF, BF, DF}
- Prime Attributes: A, B, D, F
- Non-Prime Attributes: C, E

Q5:

Relation R(ABCDE) having dependencies as:

$CE \rightarrow D$

$D \rightarrow B$

$C \rightarrow A$

Highest normal form? C.K set?

ANS:

Rules given: $CE \rightarrow D$, $D \rightarrow B$, $C \rightarrow A$

- How to find the Candidate Key: Look for letters that are never on the right side of the arrows. 'C' and 'E' are never the result of a rule, so you *must* start with them to get the full set. Let's see what happens if we start with CE:
 1. We have C and E.
 2. The rule $CE \rightarrow D$ gives us D. Now we have C, E, D.
 3. The rule $D \rightarrow B$ gives us B. Now we have C, E, D, B.
 4. The rule $C \rightarrow A$ gives us A. Now we have all of them: A, B, C, D, E. *Result:* CE is the only candidate key.
- Highest Normal Form (1NF):

- The "prime" attributes (parts of your key) are C and E. Everything else (A, B, D) is "non-prime".
- To be in Second Normal Form (2NF), you cannot have a rule where just a *piece* of your key figures out a non-prime attribute.
- Look at the rule $C \rightarrow A$. The letter 'C' is only half of your key (CE), but it acts on its own to figure out 'A'. This is a shortcut that breaks 2NF. Because it fails 2NF, it stays in 1NF.

Q6) Relation R(ABCDEF) having dependencies as:

$AB \rightarrow C$

$DC \rightarrow AE$

$E \rightarrow F$

Highest normal form? C.K set?

ANS:

Rules given: $AB \rightarrow C$, $DC \rightarrow AE$, $E \rightarrow F$

- How to find the Candidate Keys: The letters 'B' and 'D' are never on the right side of the arrows, so they must be in every key. BD alone isn't enough, so let's try combining them with other letters.
 - Try adding A (ABD): AB gives us C. Now we have A,B,C,D. DC gives us A (already have it) and E. Now we have A,B,C,D,E. Finally, E gives us F. We got everything! So ABD is a key.
 - Try adding C (BCD): The rule $DC \rightarrow AE$ means if we have C and D, we get A. Once we have A, we essentially have the ABD group from above, which we know gets everything. So BCD is also a key.
- Highest Normal Form (1NF):
 - Prime attributes: A, B, C, D. Non-prime: E, F.
 - Look at the rule $DC \rightarrow AE$. This means DC figures out E. 'DC' is only a piece of your candidate key (BCD). A piece of a key figuring out a non-prime attribute (E) breaks 2NF. Therefore, it is only in 1NF.

Q7) Relation R(ABCDEFGHI) having dependencies as:

$AB \rightarrow C$

$BD \rightarrow EF$

$AD \rightarrow GH$

$A \rightarrow I$

Highest normal form? C.K set?

ANS:

Rules given: $AB \rightarrow C$, $BD \rightarrow EF$, $AD \rightarrow GH$, $A \rightarrow I$

- How to find the Candidate Key: A, B, and D are never on the right side. Let's start with exactly those three (ABD).
 - AB gets us C.
 - BD gets us E and F.
 - AD gets us G and H.
 - A gets us I.
 - By just having A, B, and D, we trigger every single rule and get all the letters. ABD is the key.
- Highest Normal Form (1NF):
 - Prime attributes: A, B, D.
 - Every single rule given breaks 2NF. For example, look at $A \rightarrow I$. 'A' is just one third of your key, but it goes off on its own to figure out 'I'. You can't have partial keys doing all the work. It fails 2NF and stays in 1NF.

Q8) Relation R(ABCDE) having dependencies as:

$AB \rightarrow CD$

$D \rightarrow A$

$BC \rightarrow DE$

Highest normal form? C.K set?

ANS: Rules given: $AB \rightarrow CD$, $D \rightarrow A$, $BC \rightarrow DE$

- How to find the Candidate Keys: 'B' is never on the right side, so it's a mandatory starting point.
 - Test AB: AB gives us C and D. Since we now have B, C, and D, the rule $BC \rightarrow DE$ gives us E. We have everything. AB is a key.
 - Test BC: BC gives us D and E. Since we now have D, the rule $D \rightarrow A$ gives us A. We have everything. BC is a key.
 - Test BD: The rule $D \rightarrow A$ gives us A. Since we now have A and B, we trigger the first scenario (AB), which we already know gets everything. BD is a key.
- Highest Normal Form (3NF):
 - Prime attributes: A, B, C, D. Non-prime: E.
 - It passes 2NF because 'E' is the only non-prime attribute, and it is only found by using 'BC', which is a full key, not a partial one.
 - For 3NF, the rule is: the left side must be a full key, OR the right side must be a prime attribute. Look at $D \rightarrow A$. 'D' is not a full key. However, 'A' is a prime attribute. This gives it a special "free pass" for 3NF.

- It fails the stricter BCNF (which does not allow free passes for prime attributes), so it stops at 3NF.

Q9) Relation R(ABCDE) having dependencies as:

BC $\rightarrow$ ADE

D $\rightarrow$ B

Highest normal form? C.K set?

ANS:

Rules given: BC $\rightarrow$ ADE, D $\rightarrow$ B

- How to find the Candidate Keys: 'C' is never on the right side.
 - Test BC: The rule BC $\rightarrow$ ADE gives us everything immediately. BC is a key.
 - Test CD: The rule D $\rightarrow$ B means if we have D, we get B. If we have C and D, we get C and B. We already know BC gets us everything. So, CD is also a key.
- Highest Normal Form (3NF):
 - Prime attributes: B, C, D. Non-prime: A, E.
 - It passes 2NF because 'BC' (a full key) is the only thing figuring out the non-prime attributes A and E.
 - Look at D $\rightarrow$ B. 'D' is not a full key, which would normally be a problem. But 'B' is a prime attribute (it's part of the BC key). Because the right side is prime, it passes 3NF. It fails BCNF for the same reason as above.

Q10) Relation R(ABC) having dependencies as:

A $\rightarrow$ B

B $\rightarrow$ C

C $\rightarrow$ A

Highest normal form? C.K set?

ANS: Rules given: A $\rightarrow$ B, B $\rightarrow$ C, C $\rightarrow$ A

- How to find the Candidate Keys: These rules create a perfect circle.
 - Start with A: A gives B, B gives C. You get A, B, C.
 - Start with B: B gives C, C gives A. You get A, B, C.
 - Start with C: C gives A, A gives B. You get A, B, C.
 - A, B, and C are all individual candidate keys.
- Highest Normal Form (BCNF):
 - To be in Boyce-Codd Normal Form (BCNF), the absolute strictest rule applies: the left side of *every single rule* must be a full candidate key. No exceptions.
 - Look at the left sides of our rules: A, B, and C. Since A, B, and C are all full candidate keys, this table perfectly follows the rule. It is in BCNF.

Q11) Consider a relation P having fields as F1, F2, F3, F4, F5 with the functional dependencies as follows:

- F1 $\rightarrow$ F3
- F2 $\rightarrow$ F4
- (F1, F2) $\rightarrow$ F5

In terms of normalization, this table is in which normal form. Describe the prime and non-prime attributes for the relation P.

ANS:

Rules given:

- F1 $\rightarrow$ F3
- F2 $\rightarrow$ F4
- (F1, F2) $\rightarrow$ F5

Step 1: Finding the Candidate Key Look at the right side of the arrows. The fields F3, F4, and F5 are all determined by something else. However, F1 and F2 never appear on the right side. This means no other fields can be used to find F1 or F2, so they must be part of your starting key. Let's see what happens if we group them together as (F1, F2):

- We start with F1 and F2.
- The first rule F1 $\rightarrow$ F3 means having F1 gives us F3. Now we have F1, F2, F3.
- The second rule F2 $\rightarrow$ F4 means having F2 gives us F4. Now we have F1, F2, F3, F4.
- The third rule (F1, F2) $\rightarrow$ F5 means having both gives us F5. Now we have all the fields: F1, F2, F3, F4, F5.
- Result: The combination (F1, F2) is the only candidate key.

Step 2: Identifying Prime and Non-Prime Attributes

- Prime Attributes: These are the fields that make up your candidate key.
 - Prime Attributes = F1, F2
- Non-Prime Attributes: These are all the leftover fields that are not part of the key.
 - Non-Prime Attributes = F3, F4, F5

Step 3: Determining the Highest Normal Form To move beyond First Normal Form (1NF) into Second Normal Form (2NF), a strict rule must be met: you cannot use just a *piece* of your candidate key to figure out a non-prime attribute. You must use the whole key.

- Our candidate key is two parts: (F1, F2).
- Look at the rule F1 $\rightarrow$ F3. F1 is only half of our candidate key, but it acts on its own to figure out F3 (a non-prime attribute). This is known as a partial dependency.
- Because a partial dependency exists, the table fails the test for 2NF.
- Result: The highest normal form for relation P is 1NF.

Q12 Consider a relation R(ABCDEFGHI) having FDs as:

- AB $\rightarrow$ C
- AD $\rightarrow$ GH
- BD $\rightarrow$ EF
- A $\rightarrow$ I
- H $\rightarrow$ J

Identify that relation is in which normal form and the candidate key by determining the prime and non prime attributes.

ANS:

Rules given: AB $\rightarrow$ C, AD $\rightarrow$ GH, BD $\rightarrow$ EF, A $\rightarrow$ I, H $\rightarrow$ J

- How to find the Candidate Key: Look for the letters that never appear on the right side of the arrows. The letters A, B, and D are never the result of a rule, so they are mandatory starting points. Let's test them together as ABD:
 1. We start with A, B, and D.
 2. The rule AB $\rightarrow$ C gives us C. Now we have A, B, C, D.
 3. The rule AD $\rightarrow$ GH gives us G and H. Now we have A, B, C, D, G, H.
 4. The rule BD $\rightarrow$ EF gives us E and F. Now we have A, B, C, D, E, F, G, H.
 5. The rule A $\rightarrow$ I gives us I. We now have A-I.
 6. The rule H $\rightarrow$ J gives us J. We have successfully found all attributes. *Result:* ABD is the only candidate key.
- Prime and Non-Prime Attributes:
 - Prime Attributes: A, B, D (the parts of your key).
 - Non-Prime Attributes: C, E, F, G, H, I, J (everything else).
- Highest Normal Form (1NF):
 - To reach Second Normal Form (2NF), you cannot have a rule where just a *piece* of your candidate key figures out a non-prime attribute.
 - Look at the rule A $\rightarrow$ I. The letter 'A' is just one-third of your key (ABD), but it acts entirely on its own to figure out 'I'. This is a shortcut (partial dependency) that breaks the rules of 2NF.
 - Because it fails 2NF, this table is only in 1NF.

Q13) Relation R having attributes as (A, B, C, D, E) having the functional dependencies as follows:

- A $\rightarrow$ B
- B $\rightarrow$ E
- C $\rightarrow$ D

Identify that relation is in which normal form and the candidate key by determining the prime and non prime attributes.

ANS: Rules given: $A \rightarrow B$, $B \rightarrow E$, $C \rightarrow D$

- How to find the Candidate Key: Check which letters never show up on the right side of the arrows. The letters A and C are never results, so they must be in every key. Let's test them together as AC:
 1. We start with A and C.
 2. The rule $A \rightarrow B$ gives us B. Now we have A, B, C.
 3. The rule $B \rightarrow E$ gives us E. Now we have A, B, C, E.
 4. The rule $C \rightarrow D$ gives us D. Now we have all the attributes: A, B, C, D, E. *Result:* AC is the only candidate key.
- Prime and Non-Prime Attributes:
 - Prime Attributes: A, C.
 - Non-Prime Attributes: B, D, E.
- Highest Normal Form (1NF):
 - We must check for 2NF violations (where a piece of the key determines a non-prime attribute).
 - Our key is AC. Look at the rule $A \rightarrow B$. The letter 'A' is only half of the key, but it independently determines 'B' (a non-prime attribute).
 - Similarly, $C \rightarrow D$ shows the other half of the key ('C') independently figuring out 'D'.
 - Both of these are partial dependencies that break 2NF. Therefore, the relation is only in 1NF.

LEARNING OUTCOMES:

- Determine candidate keys by finding attributes absent from the right side of functional dependencies and computing their closure.
- Categorize attributes as prime if they belong to any candidate key and non-prime if they do not.
- Identify 1NF relations by spotting partial dependencies where a fraction of a key determines a non-prime attribute.
- Distinguish 3NF from BCNF by checking if a dependency relies on a prime attribute exception rather than a full superkey.

CONCLUSION:

Mastering these normalization steps provides a systematic method for evaluating database schemas against structural anomalies. By tracing functional dependencies to establish keys and scrutinizing



how all other attributes relate to those keys, you can reliably decompose flawed tables into efficient, anomaly-free relational designs.